
LECTURE NOTE

Lecture 2:

AI509 - Natural Language Processing

Simon Holm
Fall - 2026



Contents

1 Mixture of Experts (MoE) 3

2 DeepSeek Sparse Attention (DSA) 3

3 Rotary position embedding RopE 4

4 Multi-head Latent Attention 4

5 Multi-token predictions 5

6 Muon Optimizer 6

7 RMS Normalization 6

8 SwiGLU 6

1 Mixture of Experts (MoE)

From the switch transformer back in 2022. A modification in the feed forward layer where we store multiple feed forward layers (experts) and a router which learns which expert is most relevant for the input. In standard MoE, parameters are not shared across transformer layers.

1. Add a routing mechanism W_R to determine which expert is the best fit.

Probability that expert i is best suited for token representation x

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_{j=1}^N e^{h(x)_j}} \quad \text{where } h(x) = W_r x$$

1. Only the top- k experts (feedforward modules) get to process a given example
2. Outputs of these experts are aggregated according to routing scores

$$y = \sum_{I=T} p_i(x) E_i(x)$$

where E_i is expert number i and $p_i(x)$ is the estimated probability that expert I is the best fit.

2 DeepSeek Sparse Attention (DSA)

Based on the ‘lightning’ indexer. Computes the index score $I_{t,s}$ between the query token $h_t \in \mathbb{R}^d$ and a preceding token $h_s \in \mathbb{R}^d$, determining which tokens to be selected by the query token:

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \text{ReLU}(q_{t,j}^I \cdot k_{t,j}^I)$$

Then the attention output u_t is computed by applying the attention mechanism between the query token h_t and a sparsely selected key-value entries $\{c_s\}$:

$$u_t = \text{Attn}(h_t, \{c_s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\})$$

In training, both a warmup-stage (with no cutoff) is done, then afterwards a sparse-stage training with k -cutoff

3 Rotary position embedding RopE

$$f_{\{q,k\}}(x_m, m) = R_{\Theta, m}^d W_{\{q,k\}} x_m$$

where

$$R_{\Theta, m}^d = \begin{bmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \dots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \dots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{bmatrix}$$

Then

$$q_m^\top k_n = (R_{\Theta, m}^d W_q x_m)^\top (R_{\Theta, n}^d W_k x_n) = x^\top W_q R_{\Theta, n-m}^d W_k x_n$$

4 Multi-head Latent Attention

Revisit standard multi-head attention.

One could also slice one big matrix W_q into n heads $[q_1, q_2, \dots, q_n]$

Now introduce latent variables c_t^Q and c_t^{KV} . These variables allow to split queries and keys, so only some part of them are applied with RopE.

Because of the linkage between keys and values, we used cached latent variables specifically for c_t^{KV} . *you can think of the latent variables as a learnable parameter which learns which parts of the queries have position-relevant context and which have not.*

This is nice, because you then do not need to recalculate the attention for $x_{>t}$ when x_t appears in generation.

5 Multi-token predictions

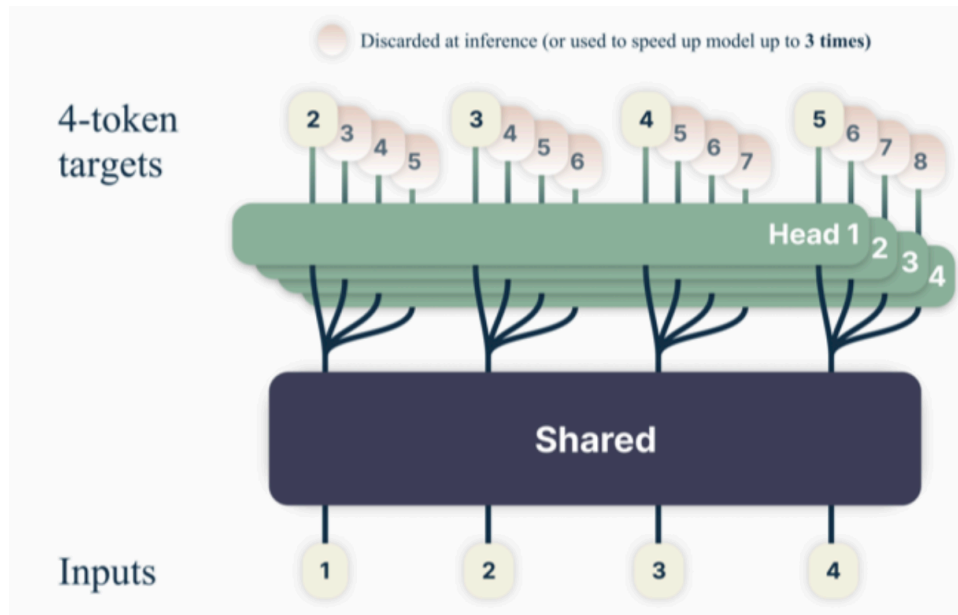


Figure 1: Multi-token predictions with multiple heads

Inventors just used 1 head since it performed good enough. GLM 5 used parameter sharing in the heads.

6 Muon Optimizer

Muon = MomentUm Orthogonalized by Newton-Schulz

Algorithm MuonClip Optimizer

```
1: for each training step  $t$  do
2:   for each weight  $W \in R^{(n \times m)}$  do
3:      $M_t = \mu M_{t-1} + G_t$ 
4:      $O_t = \text{Newton-Schulz}(M_t) \cdot \sqrt{\max(n, m)} \cdot 0.2$ 
5:      $W_t = W_{t-1} - \eta(O_t + \lambda(W_{t-1}))$ 
6:   end for
7:   for each attention head  $h$  in every attention layer of the model do
8:     Obtain  $S_{\max}^h$  already computed during forward
9:     if  $S_{\max}^h > \tau$  then
10:       $\gamma \leftarrow \tau / S_{\max}^h$ 
11:       $W_{qc}^h \leftarrow W_{qc}^h \cdot \sqrt{\gamma}$ 
12:       $W_{kc}^h \leftarrow W_{kc}^h \cdot \sqrt{\gamma}$ 
13:       $W_{qr}^h \leftarrow W_{qr}^h \cdot \gamma$ 
14:    end if
15:   end for
16: end for
```

- Muon excels at token efficiency
- But can be a bit unstable
- Therefore, every other new LLM paper proposes some small variation to Muon
- Also; Muon is only used for feedforward layer parameters. Embeddings still use AdamW.

7 RMS Normalization

Assume mean is 0 for LayerNorm, so we only normalize variance

$$y_i = \frac{x_i}{\text{RMS}(x)} \cdot \gamma_i \quad \text{where } \text{RMS}(x) = \sqrt{\varepsilon + \frac{1}{n} \sum_{i=1}^n x_i^2}$$

8 SwiGLU

A mix of Swish and GLU

$$\text{FFN}(x) = (\text{Swish}_\beta(xW_1) \cdot (xV))W_2$$